FIAP

# SPRINT 4

COMPUTATIONAL THINKING WITH PYTHON

# NEXTCODE | SMARTSTUDY

Evolução do simulador de câmera acadêmica inteligente


**Integrantes**

Kaick Lima Silva — RM 574060
Gustavo Basso — RM 572623
Guilherme Sales — RM 572933
Pedro Feltrin Geraldes — RM 569038
Guilherme Kozikoski Failla — RM 571611


São Paulo — 2026

# SUMÁRIO

1. Descritivo do projeto

2. Funcionalidades implementadas

3. Arquitetura e organização do código

4. Persistência e integração com API externa

5. Validações, exceções e boas práticas

6. Alterações em relação à Sprint 3

7. Como executar e testar

8. Limitações e considerações finais

# 1. DESCRITIVO DO PROJETO

## 1.1 Objetivo

O SmartStudy é uma solução acadêmica conceitual da equipe NextCode. Seu propósito é apoiar o registro e a organização de conteúdos de estudo por disciplina. A Sprint 4 evolui o programa de terminal da Sprint 3, acrescentando armazenamento permanente em JSON, organização modular, orientação a objetos e consulta a uma API pública de livros.

## 1.2 Justificativa

Na Sprint 3, o programa mantinha os dados em listas e dicionários durante a execução. Ao encerrar o programa, esses registros eram perdidos. A Sprint 4 trata essa limitação com persistência local. Também aproxima o protótipo da proposta SmartStudy descrita no Checkpoint 4: organizar materiais acadêmicos, validar links e apoiar atividades de estudo.

## 1.3 Escopo e transparência

O projeto continua sendo um protótipo de lógica em Python. Não controla a câmera física e não processa imagens reais. A digitalização/OCR é simulada: o usuário informa o texto reconhecido. A consulta à Open Library é real e depende de conexão. A validação de QR Code é uma verificação básica de formato e domínio permitido, não uma garantia de segurança.

# 2. FUNCIONALIDADES IMPLEMENTADAS

## 2.1 Modo Inteligente

O usuário seleciona Sala de Aula, Biblioteca ou Externo e informa a disciplina. O sistema aplica configurações simuladas de ISO, foco e ajustes. Preferências podem ser salvas por disciplina em JSON. Também é possível registrar uma captura simulada na galeria.

## 2.2 Digitalização de documento (OCR simulado)

O sistema verifica se o usuário confirma o alinhamento do documento. Em seguida, recebe nome, disciplina e texto reconhecido manualmente. Um registro PDF simulado é adicionado à galeria. Não há leitura óptica real nem criação do arquivo PDF de imagem.

## 2.3 Galeria de estudos

Permite listar registros, pesquisar por disciplina e reclassificar um registro pelo identificador. Cada item armazena tipo, disciplina, nome, ambiente, data e, quando informado, texto reconhecido. Os registros permanecem disponíveis após reiniciar o programa.

### 2.4 Verificação de QR Code

Recebe um endereço e verifica esquema HTTP/HTTPS e domínio. São permitidos domínios terminados em .edu, .gov e .org, além de fiap.com.br e seus subdomínios. A comparação considera o hostname, evitando aceitar um endereço apenas porque contém o texto 'fiap.com.br' em outro trecho. A lista é uma regra demonstrativa e não substitui análise antiphishing.

### 2.5 Grade de aulas

Permite cadastrar, visualizar e excluir aulas. Cada registro contém identificador, disciplina, dia da semana e horário. O horário é validado no formato HH:MM. Os dados são persistidos em JSON.

### 2.6 Busca acadêmica — Open Library API

O usuário informa uma disciplina ou tema. O módulo crawler consulta a API pública de pesquisa da Open Library e apresenta título, autores e ano de publicação. O usuário pode salvar os resultados no acervo local. A aplicação trata falhas HTTP, ausência de conexão, timeout e respostas JSON inválidas.

### 2.7 Flashcards

Permite criar cartões com disciplina, pergunta e resposta. Na revisão, o usuário visualiza a pergunta e solicita a revelação da resposta. Os cartões ficam armazenados no JSON. A criação é manual; não é apresentada como geração automática por IA.

## 3. ARQUITETURA E ORGANIZAÇÃO DO CÓDIGO

O código foi separado por responsabilidade para facilitar manutenção, testes e leitura. O menu principal fica em main.py e chama funções específicas de cada fluxo.

| Arquivo | Responsabilidade | Por que foi utilizado |
|---|---|---|
| main.py | Menu e navegação | Centraliza o fluxo e retorna ao menu após cada operação. |
| src/system.py | Classe SmartStudy | Agrupa regras e operações sobre os dados. |
| src/storage.py | Classe JsonStorage | Isola leitura e gravação do arquivo JSON. |
| src/validators.py | Validações | Evita duplicação de verificações de entrada. |
| src/api_crawler.py | OpenLibraryCrawler | Isola comunicação HTTP e tratamento de resposta. |

### 3.1 Orientação a objetos

A classe SmartStudy representa o núcleo das regras do sistema. Ela recebe um objeto de armazenamento e mantém os dados carregados. Seus métodos executam operações como salvar captura, consultar preferências, pesquisar galeria, cadastrar aula e criar flashcard. Essa escolha concentra comportamentos relacionados e evita espalhar manipulação direta do JSON pelo menu.

### 3.2 Subrotinas, parâmetros e retornos

As funções do menu recebem o objeto do sistema como parâmetro. Métodos recebem dados específicos — por exemplo, disciplina, horário ou identificador — e retornam resultados booleanos ou listas quando necessário. Isso permite reutilizar operações e verificar se a persistência foi concluída.

## 4. PERSISTÊNCIA E INTEGRAÇÃO COM API EXTERNA

### 4.1 Persistência JSON

O arquivo data/smartstudy.json é criado automaticamente. Ele armazena galeria, preferências, grade de aulas, referências bibliográficas e flashcards. A gravação utiliza arquivo temporário e substituição do arquivo principal,

reduzindo o risco de deixar um JSON parcialmente escrito. Se o JSON estiver inválido, o sistema tenta preservar o arquivo corrompido com outro nome e inicia uma estrutura vazia.

### 4.2 API Open Library

A integração usa o endpoint de pesquisa pública /search.json. A consulta é feita com urllib da biblioteca padrão, sem chave de API ou instalação adicional. O crawler define timeout, identifica erros HTTP e de conexão, interpreta o JSON e normaliza os campos relevantes. Os resultados são exibidos e podem ser persistidos no acervo.

## 5. VALIDAÇÕES, EXCEÇÕES E BOAS PRÁTICAS

• Campos obrigatórios não aceitam texto vazio.

• Menus aceitam somente opções previamente definidas.

• Entradas numéricas são convertidas com try/except.

• Horários precisam seguir o padrão de 24 horas HH:MM.

• Links precisam ter hostname e esquema HTTP ou HTTPS.

• A API possui timeout e tratamento de erros de rede e JSON.

• A leitura e gravação de arquivos tratam exceções de I/O.

• O código utiliza nomes descritivos e separação de responsabilidades.

• O menu oferece opção explícita para encerrar a execução.

## 6. ALTERAÇÕES EM RELAÇÃO À SPRINT 3

### 6.1 O que foi mantido

Foram mantidos o Modo Inteligente, o registro de documentos, a galeria, a verificação de QR Code e a grade de aulas. Essas funções já faziam parte do programa anterior e continuam coerentes com a proposta.

### 6.2 O que foi modificado

A galeria deixou de existir apenas em memória e passou a ser persistida. As preferências por disciplina e a grade também são salvas. A galeria ganhou pesquisa por trecho e reclassificação por ID. A verificação de links passou a analisar o hostname com urlparse e terminações de domínio, em vez de procurar apenas uma substring. A grade ganhou exclusão e validação de horário.

### 6.3 O que foi adicionado

Foram adicionados o módulo de armazenamento JSON, a classe SmartStudy, o crawler OpenLibraryCrawler, a pesquisa externa de livros e o módulo de flashcards. Também foram adicionados tratamento de falhas de rede, recuperação de JSON inválido e mensagens específicas para limitações do protótipo.

### 6.4 Justificativa das decisões

A persistência responde diretamente à necessidade de manter os dados após o encerramento. A separação em módulos facilita manutenção e demonstra subrotinas com parâmetros e retornos. A orientação a objetos agrupa as operações de negócio. A API externa atende ao requisito de consumo de serviço externo e acrescenta referências de estudo. Os flashcards aproximam o protótipo do módulo de retenção de conteúdo descrito no conceito SmartStudy. As funções de câmera e OCR permanecem explicitamente simuladas para não alegar recursos que o código não implementa.

## 7. COMO EXECUTAR E TESTAR

**Requisito:** Python 3.10 ou superior. Não é necessária instalação de bibliotecas externas.

**Execução:** abrir o terminal na pasta do projeto e executar `python main.py`.

• Cadastrar uma aula e encerrar. Abrir novamente e confirmar que a aula permanece.

• Criar uma captura simulada e localizar o registro na galeria.

• Reclassificar um registro e conferir a nova disciplina.

• Cadastrar um flashcard, encerrar e revisar após reiniciar.

• Consultar uma disciplina na Open Library com internet disponível.

• Testar um link HTTPS com domínio permitido e outro fora da lista.

• Informar horário inválido e confirmar que o sistema solicita novamente.

# 8. LIMITAÇÕES E CONSIDERAÇÕES FINAIS

A Sprint 4 entrega um protótipo Python modular com persistência local e integração externa. A API pode estar indisponível; nesse caso, os dados locais continuam acessíveis. O programa não implementa captura de câmera, OCR real, leitura física de QR Code, autenticação, sincronização em nuvem nem geração de conteúdo por IA. Essas limitações são declaradas para diferenciar funcionalidades implementadas de recursos previstos no conceito do produto.

A solução atende aos requisitos centrais da Sprint 4: menu navegável, funções com parâmetros e retornos, validações, try/except, persistência JSON, consumo de API externa e organização do código.

# ADENDO - FUNCIONALIDADES ADICIONADAS

Atualizacao da documentacao da Sprint 4

Este adendo complementa a documentacao-base sem substituir as funcionalidades ja descritas. As informacoes abaixo refletem a implementacao atual do projeto.

## 9. FLASHCARDS AUTOMATICOS COM CONCEITOS MATEMATICOS

### 9.1 Fonte externa gratuita

Foi adicionada a consulta a Wikipedia, usando a API de busca da MediaWiki e a API REST de resumo de paginas, para obter conceitos e descricoes matematicas. Essa integracao e distinta da Open Library: esta continua destinada a referencias bibliograficas, enquanto a Wikipedia fornece o texto-base dos conceitos.

### 9.2 Selecao e geracao de cartoes

O menu de flashcards apresenta 16 materias matematicas. A materia pode ser informada pelo nome ou pelo numero da lista; a entrada textual aceita aproximacoes para tolerar erros de digitacao. Ao selecionar uma materia, o programa consulta a API e cria 10 flashcards com perguntas e respostas, que sao persistidos no arquivo JSON local. A opcao de revisao embaralha os cartoes e revela a resposta apos a confirmacao do usuario. Durante a escolha da materia, e possivel cancelar digitando "sair" ou "0". A criacao manual de flashcards permanece disponivel.

### 9.3 Quiz - situacao atual

O modulo de regras tambem possui um gerador de 10 questoes de multipla escolha, com alternativas, resposta correta, explicacao e ordem aleatoria. Contudo, a versao atual do menu principal nao oferece uma opcao para iniciar esse quiz. Portanto, essa capacidade esta implementada no nucleo do sistema, mas ainda nao esta acessivel como fluxo de uso pela interface de terminal.

### 9.4 Como testar

Execute **python main.py**, escolha **7 - Criar e revisar flashcards** e depois **3 - Criar com conceito matematico**. Informe um nome de materia (ou selecione seu numero), aguarde a consulta externa e confirme a criacao dos 10 cartoes. Para revisao, escolha a opcao 2. A geracao exige conexao com a internet; os cartoes gerados ficam salvos localmente.

### 9.5 Resultados e limites

Na validacao da implementacao, a consulta de Algebra retornou resumo e permitiu gerar 10 flashcards e 10 questoes no gerador interno. A resolucao aproximada de "algebr" tambem identificou Algebra. O conteudo depende da disponibilidade e da qualidade dos resumos da Wikipedia; as perguntas sao montadas por modelos de texto e nao constituem material curricular revisado por especialistas. A integracao externa pode falhar por indisponibilidade, conexao ou limitacao de requisicoes.

Este adendo atualiza a descricao anterior de flashcards exclusivamente manuais: a criacao manual foi mantida e a geracao automatica por conceito matematico foi acrescentada. A declaracao de que o sistema nao gera conteudo por IA continua correta; os cartoes e questoes sao montados por regras e texto da API, sem modelo de IA.